

K-Means Clustering Algorithm (Machine Learning)

1. Type of Learning

- **Supervised Learning:** Data has labels/target variables (we know the answer).
 - **Unsupervised Learning:** Data **does not have labels** (we don't know the answer).
 - Goal: Find hidden patterns or clusters in the data.
 - **K-Means** is a popular algorithm for clustering.
-

2. What is K-Means?

- K-Means groups data points into **K clusters**.
- **K** = number of clusters you want (must be decided before running the algorithm).
- Each cluster has a **centroid** (center point).

Steps in K-Means:

1. Start with K random points as **centroids**.
 2. Assign each data point to the **nearest centroid** → forms initial clusters.
 3. **Recalculate centroids** based on current clusters (like finding the center of gravity).
 4. Repeat steps 2–3 until **clusters no longer change**.
 5. Final clusters are formed.
-

3. Choosing K (Number of Clusters)

- Hard to guess K in real datasets.
 - Use the **Elbow Method**:
 - a. Try different K values (e.g., 1, 2, 3...10).
 - b. Calculate **Sum of Squared Errors (SSE)** for each K:
 - $SSE = \text{sum of (distance from each point to its centroid)}^2$
 - c. Plot K vs. SSE – find the “**elbow**” **point** where SSE decrease slows down.
 - d. That K is usually the **best choice**.
-

4. Python Implementation

Step 1: Import Data

```
import pandas as pd
df = pd.read_csv("data.csv")
```

Step 2: Visualize Data

```
import matplotlib.pyplot as plt
plt.scatter(df['age'], df['income'])
plt.show()
```

Step 3: Scale Features

- Scaling ensures features like income (large values) and age (small values) are comparable.

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[['age', 'income']] = scaler.fit_transform(df[['age', 'income']])
```

Step 4: Apply K-Means

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3) # choose K
y_pred = kmeans.fit_predict(df[['age', 'income']])
df['cluster'] = y_pred
```

Step 5: Visualize Clusters

```
plt.scatter(df[df['cluster']==0]['age'], df[df['cluster']==0]
['income'], color='red')
plt.scatter(df[df['cluster']==1]['age'], df[df['cluster']==1]
['income'], color='green')
plt.scatter(df[df['cluster']==2]['age'], df[df['cluster']==2]
['income'], color='blue')
plt.scatter(kmeans.cluster_centers_[0,0],
kmeans.cluster_centers_[0,1], color='black', marker='x', s=100) #
centroids
plt.show()
```

5. Key Points

- K-Means is **unsupervised** → works without labels.
- K-Means needs **K clusters** in advance.
- **Centroids** are recalculated until clusters stabilize.
- **Feature scaling** is important to get correct clusters.

- Use **Elbow Method** to choose the best K.
-

6. Exercise

- Dataset: Iris flowers
 - Features: Petal length and width
 - Steps:
 - a. Ignore target labels
 - b. Scale features
 - c. Apply K-Means
 - d. Use Elbow Method to find optimal K
 - e. Plot clusters
-

2. Why Scaling is Important

- Scaling puts all features on the same scale usually 0 to 1 (Min-Max scaling)
- with zero mean & unit variance (standardization)
- After scaling, all features contribute equally to distance calculation.
- K-Means can then find true clusters based on all features, not just the one with large numbers.

3. How to Scale

• Min-max Scaling

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

df[['Age', 'Income (\$)']] = scaler.fit_transform(df[['Age', 'Income (\$)']])

→ converts all values 0-1 range

→ keeps relative differences intact

• Standardization (Alternative)

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

df[['Age', 'Income (\$)']] = scaler.fit_transform(df[['Age', 'Income (\$)']])

→ converts values to mean = 0, std = 1

→ useful if features have outliers or very different distributions

2. Why Scaling is Important

- Scaling puts all features on the same scale usually 0 to 1 (Min-Max scaling)
- with zero mean & unit variance (standardization)
- After scaling, all features contribute equally to distance calculation.
- K-Means can then find true clusters based on all features, not just the one with large numbers.

3. How to Scale

• Min-max Scaling

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

df[['Age', 'Income (\$)']] = scaler.fit_transform(df[['Age', 'Income (\$)']])

→ converts all values 0-1 range

→ keeps relative differences intact

• Standardization (Alternative)

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

df[['Age', 'Income (\$)']] = scaler.fit_transform(df[['Age', 'Income (\$)']])

→ converts values to mean = 0, std = 1

→ useful if features have outliers or very different distributions